

Deep Learning - Lecture 1

Kalinin Nikita

Contents

Core Concepts	2
Mathematical Notation and Definitions	2
Alternatives to Backpropagation	2
Historic Approaches	2
Modern Approaches	3
Gradient Calculation for Neural Networks	3
Backpropagation	3
Symbolic vs Numeric Differentiation	4
Computational Graph	4
Computational Graph Example	4
Forward Pass	4
Backward Pass	5
Neural Network Layers	5
Neural Network Architectures	5
Layer Calculations	6
Activation Functions and Vanishing Gradients	6
1. Sigmoid Activation Function	6
2. ReLU (Rectified Linear Unit)	7
3. Leaky ReLU	8

Core Concepts

- **Backpropagation:** A key algorithm for training neural networks by propagating the error backwards through the network to adjust the weights.
- **Fully-connected Networks:** Each neuron is connected to every neuron in the previous layer, making them suitable for tasks like classification.

Mathematical Notation and Definitions

Let the input features be represented as:

$$\{(x_i, y_i)\}_{i=1}^l$$

where $x_i \in \mathbb{R}^d$ is an input vector, and y_i is the corresponding target value. Here:

- $x \in \mathbb{R}^d$: Input vector with d dimensions.
- $\mathbf{w} \in \mathcal{W}$: Weight vector of the neural network.

The network's output for an input x can be written as:

$$a(x) = w_1x_1 + \dots + w_dx_d = f(x, \mathbf{w})$$

Given a training set, the goal is to minimize the loss function $L(\hat{y}, y)$, where:

$$Q(\mathbf{w}) = \frac{1}{l} \sum_{i=1}^l L(f(x_i, \mathbf{w}), y_i)$$

This represents the average loss across all samples in the training set. The aim is to find the optimal weights $\mathbf{w}$ that minimize $Q(\mathbf{w})$.

Alternatives to Backpropagation

Historically, other methods have been explored as alternatives to backpropagation:

Historic Approaches

- **Evolutionary Algorithms (1950s)**[\[1\]](#): Optimization methods inspired by natural selection, often computationally expensive for large networks.
- **Reservoir Computing (2000)**[\[2\]](#): Uses fixed random weights in a large recurrent neural network (RNN), making it useful for certain tasks, especially where the input is sequential.

Modern Approaches

- **Direct Feedback Alignment (2016)**[4]: A variant of backpropagation where the feedback path does not use the exact gradients but rather a fixed random path.
- **Forward-Forward Algorithm (2022)**[3]: A recent approach that aims to replace backpropagation by only using forward passes, potentially reducing the complexity and computational cost of training.

Gradient Calculation for Neural Networks

In neural networks, the output y is often a function of intermediate layers:

$$y = f(g(x))$$

The gradient can be calculated as:

$$\frac{dy}{dx} = \frac{dy}{df} \cdot \frac{dg}{dx}$$

where $g(x)$ represents an intermediate transformation of the input x , and $\frac{dy}{df}$ is calculated based on the loss with respect to the output layer.

Backpropagation

1. For a composition of functions, let:

$$y = f(g(x))$$

Then the derivative $\frac{dy}{dx}$ is given by the chain rule:

$$\frac{dy}{dx} = \frac{df}{dg} \cdot \frac{dg}{dx}$$

2. More generally, if $y = f(g_1(x), g_2(x), \dots, g_n(x))$, then:

$$\frac{dy}{dx} = \sum_{i=1}^n D_i f \cdot \frac{dg_i}{dx}$$

where:

$$D_i f = \frac{\partial f}{\partial z_i}$$

and $f(z_1, \dots, z_n)$ represents a function of multiple variables z_i .

Symbolic vs Numeric Differentiation

- **Symbolic Differentiation:** For expressions like x^2 , differentiation yields $2x$ for all x .
- **Numeric Differentiation:** For expressions like x^2 with a specific value $x := 3$, differentiation yields a concrete result, e.g., 6.

Computational Graph

Consider the function:

$$f = x_1^2(x_2x_3 - 2x_1^2) = x_1^2x_2x_3 - 2x_1^4$$

Calculating partial derivatives with respect to each x_i :

$$\begin{aligned}\frac{\partial f}{\partial x_1} &= 2x_1x_2x_3 - 8x_1^3 = 4 \quad (\text{when } x = (1, 2, 3)), \\ \frac{\partial f}{\partial x_2} &= x_1^2x_3 = 3, \\ \frac{\partial f}{\partial x_3} &= x_1^2x_2 = 2.\end{aligned}$$

Given $x = (1, 2, 3)$, we can calculate y as:

$$y = x_1^2(x_2x_3 - 2x_1^2)$$

Substituting values:

$$y = 1^2(2 \cdot 3 - 2 \cdot 1^2) = 1 \cdot (6 - 2) = 4$$

Computational Graph Example

For the function:

$$y = z_1z_4, \quad \text{where} \quad z_1 = x_1^2, \quad z_2 = x_2x_3, \quad z_3 = 2z_1, \quad z_4 = z_2 - z_3$$

Forward Pass

$$\begin{aligned}z_1 &= x_1^2, \\ z_2 &= x_2x_3, \\ z_3 &= 2z_1, \\ z_4 &= z_2 - z_3, \\ y &= z_1z_4.\end{aligned}$$

Backward Pass

Using the chain rule for each variable:

$$\frac{dy}{dy} = 1$$

$$\frac{dy}{dz_4} = \frac{dy}{dy} \cdot z_1 = 1 \cdot z_1 = 1,$$

$$\frac{dy}{dz_3} = \frac{dy}{dz_4} \cdot \frac{dz_4}{dz_3} = 1 \cdot (-1) = -1,$$

$$\frac{dy}{dz_2} = z_4 \cdot 1 + \frac{dy}{dz_3} \cdot \frac{dz_3}{dz_2} = 4 + (-1) \cdot 2 = 2.$$

Finally, for x :

$$\frac{dy}{dx_1} = \frac{dy}{dz_1} \cdot \frac{dz_1}{dx_1}, \quad \frac{dy}{dx_2} = \frac{dy}{dz_2} \cdot \frac{dz_2}{dx_2}, \quad \frac{dy}{dx_3} = \frac{dy}{dz_2} \cdot \frac{dz_2}{dx_3}.$$

Neural Network Layers

Let $x \in \mathbb{R}^d$ and $w \in \mathbb{R}^d$ be the input and weight vectors:

$$a(x) = \langle w, x \rangle = w^T x.$$

Consider a multi-layer network:

$$z_1 = W_1 x + b_1, \quad z_2 = W_2 z_1 + b_2, \dots, \quad z_n = W_n z_{n-1} + b_n$$

where $W_i \in \mathbb{R}^{d_i \times d_{i-1}}$ and $b_i \in \mathbb{R}^{d_i}$.

The output layer can be represented as:

$$z_n = W_n \dots W_2 W_1 x + b$$

with weights $W \in \mathbb{R}^{d_n \times d}$.

Let $\sigma(\cdot)$ be the activation function:

$$z_i = \sigma(W_i z_{i-1} + b_i).$$

Neural Network Architectures

A **dense layer** (also known as a **fully-connected layer**) computes a latent representation z_i as a combination of inputs from the previous layer.

$$z_i = \left(z_i^{(1)}, \dots, z_i^{(d_i)} \right)$$

Each neuron in the layer is connected to all neurons in the previous layer.

MLP (Multi-Layer Perceptron): A type of fully-connected network with one or more hidden layers. **FCN** (Fully Connected Network): Another term for a neural network with fully-connected layers.

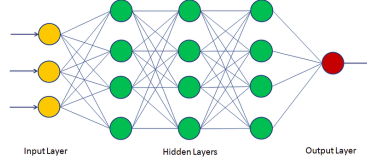


Figure 1: Multi-Layer Perceptron

Layer Calculations

In a typical layer, we have:

$$u_i = W_i z_i + b_i, \quad z_{i+1} = \sigma(u_i)$$

The gradient of the error E with respect to z_i can be computed as:

$$\frac{dE}{dz_i} = \frac{dE}{dz_{i+1}} \cdot \sigma'(u_i) \frac{du_i}{dz_i}$$

where $\sigma'(u_i)$ can become very small, contributing to the vanishing gradient issue.

Activation Functions and Vanishing Gradients

One common issue in training deep neural networks is the **vanishing gradient problem**. It occurs when gradients diminish as they propagate back through the layers, which can significantly slow down or even halt the learning process in earlier layers.

1. Sigmoid Activation Function

The sigmoid function is defined as:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Its derivative, important for gradient calculations in backpropagation, is:

$$\sigma'(x) = \sigma(x)(1 - \sigma(x)) = \frac{e^{-x}}{(1 + e^{-x})^2}$$

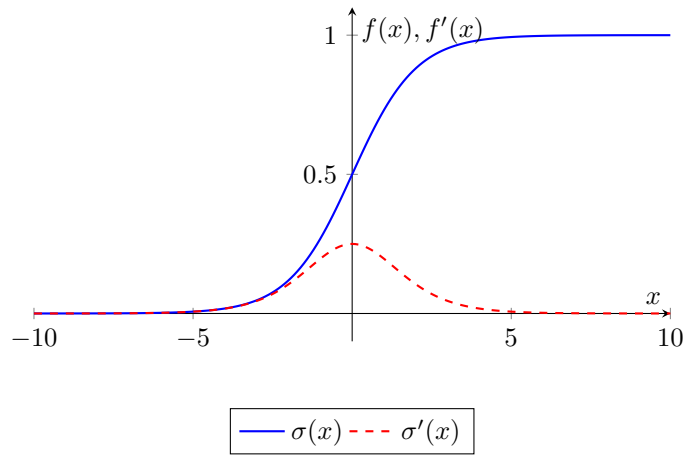


Figure 2: Sigmoid activation function and its derivative

2. ReLU (Rectified Linear Unit)

The ReLU function is defined as:

$$\text{ReLU}(x) = \max(0, x)$$

The derivative of ReLU, which is 1 for positive inputs and 0 for negative inputs, helps to mitigate the vanishing gradient problem by allowing gradients to propagate effectively for positive activations.

$$\text{ReLU}'(x) = \begin{cases} 1, & x > 0 \\ 0, & x \leq 0 \end{cases}$$

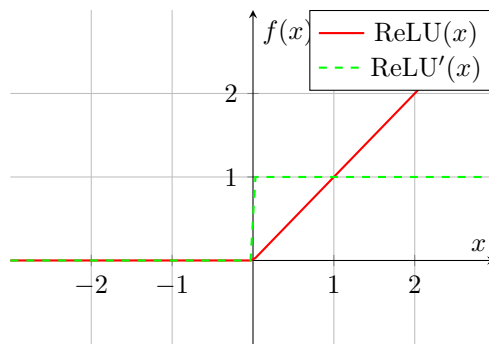


Figure 3: ReLU activation function

3. Leaky ReLU

Leaky ReLU introduces a small slope α for negative values, defined as:

$$\text{Leaky ReLU}(x) = \max(\alpha x, x)$$

This small gradient for negative inputs (controlled by α) ensures that neurons do not become inactive during training.

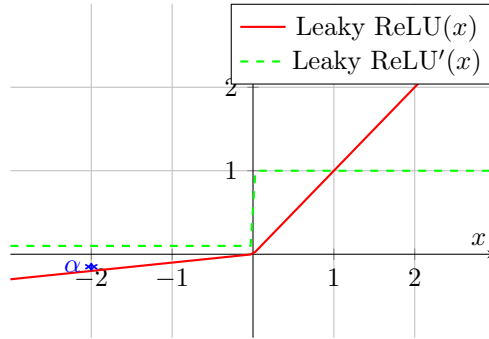


Figure 4: Leaky ReLU activation function

References

- [1] Lee Altenberg. *Evolutionary Computation*. https://dynamics.org/Altenberg/FILES/Altenberg_Evolutionary-Computation.2016.pdf?utm_source=chatgpt.com. Accessed: 2024-11-23. 2016.
- [2] Jonathan Dong et al. *Reservoir Computing Meets Recurrent Kernels and Structured Transforms*. <https://proceedings.neurips.cc/paper/2020/file/c348616cd8a86ee661c7c98800678fad-Paper.pdf>. Accessed: 2024-11-23. 2020.
- [3] Geoffrey Hinton. “The Forward-Forward Algorithm: Some Preliminary Investigations”. In: *arXiv preprint arXiv:2212.13345* (2022). URL: <https://arxiv.org/abs/2212.13345>.
- [4] Arild Nøkland. “Direct Feedback Alignment Provides Learning in Deep Neural Networks”. In: *arXiv preprint arXiv:1609.01596* (2016). URL: <https://arxiv.org/abs/1609.01596>.